

Accso Technical Interview Assignment

You are a software consultant joining a client engagement for an e-commerce platform that ships orders through multiple courier partners.

The client has a recurring integrity problem: downstream systems disagree about shipment state because courier events arrive late, out of order, duplicated, or with conflicting data. Customer support, order tracking, and incident response need a trustworthy view of what happened and what the current shipment status is.

Your task is to act as the technical lead for this problem and produce:

1. A concise technical strategy a client engineering lead would actually use
2. A pragmatic delivery plan
3. One thin executable slice that proves the approach works

This is intentionally not a full platform build. We are testing consulting and architectural judgment: how you frame the problem, surface uncertainty, make trade-offs, plan delivery, and adapt when requirements change.

Working approach

Treat this like a real consulting engagement:

- Ask clarifying questions where the brief is ambiguous, and document the questions you would ask before committing a team
- State assumptions explicitly
- Explain trade-offs in practical terms for a client engineering lead
- Optimize for a credible, well-reasoned strategy over implementation breadth

We encourage AI coding assistants and agentic tooling. We will evaluate how you used them: where they helped, where they were weak or wrong, what you changed or rejected, and how you stayed in control of the work. Be specific. A concrete example of overriding an AI suggestion is much more useful than a generic statement that you used AI.

Target roughly **3–5 hours** of focused work. Put most of that time into the strategy memo and delivery plan. The executable slice should stay thin and targeted. If you spend significantly more or less time, mention it in your notes.

Domain context

The client receives shipment events from courier partners via webhook. Internal systems need:

1. A reliable current view of each shipment
2. A queryable history of what happened and how that current view was derived

Example inbound event:

```
{
  "eventId": "evt-123",
  "partner": "dhl",
  "shipmentId": "ship-456",
  "status": "IN_TRANSIT",
  "occurredAt": "2026-03-10T12:00:00Z",
  "receivedAt": "2026-03-10T12:00:05Z",
  "location": "Amsterdam"
}
```

You may adjust this schema if a different shape better serves the problem. If you do, explain why.

Suggested status values:

- LABEL_CREATED
- HANDED_TO_CARRIER
- IN_TRANSIT
- OUT_FOR_DELIVERY
- DELIVERED
- DELIVERY_EXCEPTION
- RETURNED

The core problem is data integrity, not feature breadth. Your strategy should address duplicates, out-of-order events, and conflicting updates directly.

Required artifacts

1. Technical strategy memo

Write a concise memo (roughly 3–5 pages, not 15) covering:

- Problem framing and scope: what you are solving now, and what you are not
- Key assumptions and open questions for the client
- Proposed architecture: component boundaries, data flow, and where the hard decisions live
- Data integrity strategy for duplicates, out-of-order events, and conflicting updates
- Operational concerns: observability, failure handling, rollout approach, and ownership boundaries

Write for a technical audience that also makes business decisions. Focus on trade-offs, not just technical choices.

2. Delivery plan

Describe a phased path from nothing to a reliable first production release. Include:

- Phase breakdown: what ships first, what comes next, and what is explicitly postponed
- The minimum credible first production slice
- A problem-specific risk register with mitigation strategies
- Success signals: metrics or operational indicators that show the system is working

3. One executable slice

Implement one small but meaningful slice of the system. Choose something that demonstrates judgment, for example:

- Event normalization and deduplication logic
- Current-state derivation from an event stream
- An ingestion endpoint with persistence and tests
- A small rules engine for conflict resolution

It should be executable and testable, but it does not need to be a complete service. Its purpose is to prove the strategy in code.

Mandatory change request

After producing your initial strategy, **commit it to git before continuing**. Then apply this change:

The client now wants to onboard a second courier within one quarter. This partner sends batched events (not one at a time), frequently out of order. Legal requires raw partner payloads to be deleted after 30 days, while derived audit decisions must remain queryable for 1 year.

Update your strategy and, where sensible, your executable slice. Explain:

- How the change affects your architecture, delivery plan, and ownership model
- Which existing decisions become risky or invalid
- What new risks this introduces and how you would mitigate them
- What you would push back on or negotiate with the client

Deliverables

Submit a repository containing:

1. Your technical strategy memo (Markdown or PDF)
2. A high-level architecture or sequence diagram
3. A phased delivery plan with a risk register
4. One executable slice with tests and clear local run instructions
5. Two short ADRs covering your most important strategic decisions, including alternatives considered and why you chose your approach
6. A development process note describing how you used AI tools, what they produced, what you overrode or verified yourself, and at least one concrete example where your judgment differed from the AI's suggestion

Optional bonus: Include a short stakeholder note (about half a page) describing the trade-offs you would surface to a client sponsor or engineering manager who is not deep in the technical details.

Scope and packaging

Keep the submission lightweight. Clear local run instructions for the executable slice are required. Containerization, CI, or OpenAPI are optional unless they directly support your approach.

Do not build a complete platform or add artifacts just to look complete. A clean repository with clear documents is better than a sprawling one with impressive-looking but hollow infrastructure.

Follow-up discussion

In the interview, expect to discuss:

- Your assumptions and where the brief was underspecified
- What you would clarify with the client before committing a team
- Why you structured the delivery plan the way you did
- How your architecture and plan changed after the mandatory change request
- What you would postpone even if the client pushed for more scope
- Where the architecture is most likely to fail under real organizational constraints
- What AI accelerated and what still required your judgment
- How you would present the strategy to a non-technical client stakeholder